

Day 3 Assignment — Understanding LLMs, Tokens and Context Windows

1. Introduction

Large Language Models (LLMs) are one of the main technologies behind modern Generative AI applications. Before working with LLM APIs, it is important to understand how models receive information, process it, and generate responses.

In this assignment, I learned about **tokens, token usage, context windows, input, output, model limitations, and the effect of instructions on AI responses.**

2. What is Generative AI?

Generative AI is a type of artificial intelligence that can create new content based on the instructions provided by a user.

It can generate:

- Text
- Images
- Code
- Audio
- Video
- Summaries
- Translations

For example, if I ask an AI model:

"Write a Python function that calculates the average of three numbers."

The model can generate the Python code as a response.

In simple words: Generative AI learns patterns from large amounts of data and uses those patterns to generate new content based on a user's input.

3. What is an LLM?

An **LLM (Large Language Model)** is an AI model designed to understand and generate human language.

LLMs are trained on large amounts of text and learn patterns between words, phrases, concepts, and instructions.

Examples include:

- GPT models
- Gemini
- Claude
- Llama
- Mistral

For example, when I ask:

"Explain React.js in simple words."

The LLM processes my request and generates an explanation based on its learned language patterns.

Simple definition:

An LLM is an AI model that can understand and generate text by processing language as tokens.

4. What Are Tokens?

LLMs do not process text exactly the same way humans read it. Before processing text, the text is divided into smaller units called **tokens**.

A token can represent:

- A complete word
- Part of a word
- Punctuation
- Numbers
- Other pieces of text

For example:

I love programming!

may be broken into several tokens rather than treating the entire sentence as one unit.

Importantly:

One word does not always equal one token.

For example, a common short word might be represented by one token, while a less common or longer word may be split into multiple tokens.

Example

Generative Artificial Intelligence

The model may represent this text using multiple tokens.

The exact tokenization depends on the particular model and tokenizer.

5. Why Do Tokens Matter?

Tokens are important when developing applications with LLMs because they affect several things.

5.1 API Cost

Many commercial LLM APIs calculate usage based partly on the number of input and output tokens.

For example:

Input tokens + Output tokens = Total token usage

If an application sends a very large amount of text to an API, token usage can increase.

5.2 Context Limit

Models have limits on how much information they can process within a particular context.

Therefore, sending extremely large documents or conversations can become a problem.

5.3 Processing

More tokens generally mean more information for the model to process.

This is important when building:

- Chatbots
 - RAG systems
 - AI agents
 - Document assistants
 - Coding assistants
-

6. What Is a Context Window?

A **context window** is the amount of information an LLM can consider within a particular interaction.

The context can include things such as:

- System instructions
- User messages
- Previous conversation
- Retrieved documents
- Tool results
- Other relevant information

For example:

System instructions
+
User question
+
Previous conversation
+
Retrieved document
=
Model context

The model has a maximum context capacity determined by the particular model.

Simple Example

Imagine an AI assistant that is helping analyze a document.

If the document is small, we may be able to provide a large portion of it directly to the model.

However, if the document is extremely large, sending everything in one request may exceed the model's available context or become inefficient.

This is one reason techniques such as **chunking** and **RAG** are important.

7. Context Windows and RAG

RAG stands for **Retrieval-Augmented Generation**.

Instead of sending an entire large knowledge base to an LLM, a RAG system can:

1. Store documents.
2. Split documents into smaller chunks.
3. Create embeddings.
4. Store those embeddings in a vector database.
5. Search for relevant information when the user asks a question.
6. Send only the relevant information to the LLM.
7. Generate the answer.

Example

Suppose a company has a:

500-page employee handbook

A user asks:

"How many annual leave days do employees receive?"

A RAG system can retrieve the relevant section instead of sending all 500 pages to the LLM.

This can reduce unnecessary context and help the model focus on relevant information.

8. Input and Output

Two basic concepts when working with LLM APIs are **input** and **output**.

Input

Input is the information that we send to the model.

For example:

Explain JavaScript promises in simple words.

This is the input.

Output

Output is the response generated by the model.

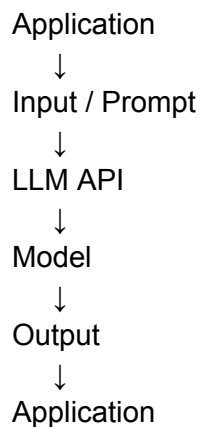
For example:

A Promise in JavaScript represents a value that may become available now, later, or never.

This is the output.

Software Developer Perspective

When using an LLM API:



This concept will become important when developing AI applications using APIs.

9. Basic Limitations of LLMs

LLMs are powerful, but they are not perfect.

Some common limitations include:

9.1 Incorrect Information

An LLM can sometimes generate an answer that sounds correct but contains incorrect information.

9.2 Hallucinations

A hallucination occurs when an AI model generates information that is unsupported, fabricated, or incorrect.

For example, an LLM might provide a citation or fact that does not actually exist.

9.3 Misunderstanding Instructions

If a prompt is unclear, the model may interpret it differently from what the user intended.

9.4 Inconsistent Responses

The same question can sometimes produce different responses depending on the prompt, context, model, settings, or conversation history.

9.5 Knowledge Limitations

An LLM may not automatically know current information unless it has access to updated information through appropriate tools or retrieval.

10. Practical Experiment

Experiment Objective

The purpose of this experiment was to observe how changing instructions can change the response generated by an LLM.

I used the same technical topic but changed the target audience.

Question 1 — Software Engineer

Prompt:

Explain REST API to a software engineer. Include technical terminology and explain HTTP methods, endpoints, status codes, and JSON responses.

Example Response:

A REST API is an HTTP-based interface that allows clients and servers to communicate using resources represented by URLs. Common HTTP methods include GET for retrieving resources, POST for creating resources, PUT/PATCH for updating resources, and DELETE for removing resources. APIs commonly return JSON representations of resources along with HTTP status codes such as 200, 201, 400, 401, 404, and 500.

Question 2 — 10-Year-Old Student

Prompt:

Explain REST API to a 10-year-old student. Avoid technical terminology and use a simple real-life example.

Example Response:

Imagine a restaurant. You are the customer, and you want to order food. You tell the waiter what you want. The waiter takes your request to the kitchen and brings the food back to you.

A REST API works in a similar way. Your application asks another application for information, and the API carries the request and brings back the result.